

1



llama-server

Quick-Start Guide for Lab Members

Model: `Qwen3.5-35B-A3B-MXFP4_MOE_BF16.gguf`

Platform: RTX 5090 • ~150 tokens/s • MXFP4 / BF16 MoE

Generated from eval_socraticlm.py • AI Lab • 2025

1. What Is llama-server?

llama-server is a lightweight HTTP server included in the llama.cpp project. It loads a large-language model (LLM) in GGUF format once into GPU memory, then stays running and listens for inference requests. Your Python scripts (or any HTTP client) can send a prompt and receive the model's reply without reloading weights every time — making batch evaluation and interactive chat very fast.

 **Note:** The model we use is Qwen3.5-35B-A3B-MXFP4_MOE_BF16.gguf. It is a 35B Mixture-of-Experts model — massive by normal standards — but runs at ~150 tokens/s on an RTX 5090 thanks to a specialised MXFP4 / BF16 MoE quantisation that packs the model efficiently without sacrificing much quality.

2. Prerequisites

- The llama.cpp binary is already compiled at `/home/ai/notebook/llama.cpp/build/bin/llama-server`
- The GGUF model file is at `/home/ai/notebook/Qwen3.5-35B-A3B-MXFP4_MOE_BF16.gguf`
- Python 3.10+ is available with the requests library (pip install requests)
- Port 8080 is free (check with: `ss -tlnp | grep 8080`)

3. Starting the Server

Open a terminal, navigate to `/home/ai/notebook`, and run the command below. Keep this terminal open while you are working — the server must stay running.

```
cd /home/ai/notebook

./llama.cpp/build/bin/llama-server \
-m Qwen3.5-35B-A3B-MXFP4_MOE_BF16.gguf \
-ngl 99 \
-fa on \
-np 1 \
-mg 0 \
-sm none \
--host 0.0.0.0 \
--port 8080 \
--jinja \
-ctk q8_0 \
-ctv q8_0 \
-b 2048 \
-ub 512 \
--temp 1.0 \
--top-p 0.95 \
--top-k 40 \
--metrics
```

 **Note:** Wait until you see a line like `llm_load_tensors: ... model loaded` or `srv start: HTTP server listening on 0.0.0.0:8080` before running your script. First load can take 10–30 seconds.

3.1 Server Flag Reference

Flag	Meaning
-m <file>	Path to the GGUF model file to load.
-ngl 99	Offload 99 (all) transformer layers to the GPU — maximises speed.
-fa on	Enable Flash Attention — reduces VRAM usage for long contexts.
-np 1	Number of parallel inference slots (1 = single request at a time).
-mg 0	Main GPU index (GPU 0 = first/only GPU).
-sm none	Split mode: no tensor split across multiple GPUs.
--host 0.0.0.0	Accept connections from any IP on the network, not just localhost.
--port 8080	TCP port the server listens on.
--jinja	Use Jinja2 for chat template formatting (required for chat models).
-ctk q8_0 / -ctv q8_0	Quantise the KV-cache to 8-bit integers — saves VRAM.
-b 2048 / -ub 512	Batch size and micro-batch size for prompt processing.
--temp / --top-p / --top-k	Default sampling parameters for generation.
--metrics	Expose Prometheus metrics at /metrics (useful for monitoring).

4. Verifying the Server Is Running

In a new terminal (or browser) run:

```
curl http://localhost:8080/health
```

You should see `{"status":"ok"}` — the server is ready.

5. Calling the Server from Python

5.1 Minimal Example (copy–paste ready)

Save this as `test_llm.py` and run it with `python3 test_llm.py`. The server must already be running in another terminal.

Remote server=140.115.126.111:8080

```
import requests

SERVER_URL = "http://localhost:8080" # change to server IP if remote

def ask(prompt: str, max_tokens: int = 256) -> str:
    payload = {
        "prompt": prompt,
        "n_predict": max_tokens,
        "temperature": 0.7,
        "top_p": 0.95,
        "stream": False,
    }
    response = requests.post(f"{SERVER_URL}/completion", json=payload, timeout=120)
    response.raise_for_status()
    return response.json()["content"].strip()

if __name__ == "__main__":
    reply = ask("Explain Newton's second law in simple terms.")
    print("Model says:", reply)
```

 **Tip:** The `/completion` endpoint accepts a raw text prompt. If you want OpenAI-style chat (messages list), use `/v1/chat/completions` instead — the server supports both.

5.2 OpenAI-compatible Chat Endpoint

llama-server also supports the OpenAI chat format, so you can use the official `openai` Python library pointing at your local server:

```
pip install openai # one-time
```

```
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="not-needed",
)

chat = client.chat.completions.create(
    model="local",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is the capital of France?"},
    ],
    max_tokens=128,
    temperature=0.7,
)

print(chat.choices[0].message.content)
```

5.3 Keeping the Server Alive for Many Requests

Because the model is loaded once, you can send thousands of requests without reloading. Reuse the same `requests.Session()` for efficiency:

```
import requests

session = requests.Session()
SERVER_URL = "http://localhost:8080"

prompts = ["What is 2+2?", "Explain gravity.", "Write a haiku about Python."]

for p in prompts:
    payload = {"prompt": p, "n_predict": 128, "temperature": 0.7, "stream": False}
    result = session.post(f"{SERVER_URL}/completion", json=payload, timeout=120)
    print(result.json()["content"].strip(), "\n---")
```

6. Accessing the Server from Another Machine

The flag `--host 0.0.0.0` means the server accepts requests from any computer on the same network. Replace `localhost` with the server's IP address in your Python script:

```
SERVER_URL = "http://192.168.1.xxx:8080"    # use the actual server IP
```

You can find the server's IP with: `ip addr show` or `hostname -I`

 **Security:** The server has no built-in authentication. Only use it on a trusted local network or set up an SSH tunnel for security.

7. Stopping the Server

Go to the terminal where the server is running and press Ctrl+C. The server will shut down gracefully and free GPU memory.

8. Quick Troubleshooting

Problem	Fix
Connection refused	Server is not running or port mismatch. Check terminal. Verify port with <code>ss -tlnp grep 8080</code>
curl returns {"status":"loading"}	Model is still loading into VRAM. Wait a few more seconds and retry.
CUDA out of memory	Another process is using the GPU. Run <code>nvidia-smi</code> to check, then free memory.
Model file not found	Check the <code>-m</code> path. Run the server command from <code>/home/ai/notebook</code> exactly as shown.
Very slow generation	Make sure <code>-ngl 99</code> is set so all layers are on GPU, not CPU.

9. Reference: LlamaServer Class (eval_socraticlm.py)

The lab's evaluation script wraps the entire server lifecycle in a Python context manager. Below is the full annotated source, followed by a usage example.

9.1 Constructor — `__init__`

All server parameters are set here. Sensible defaults are provided for every argument except `model_path`.

```
class LlamaServer:
    """Manages a llama-server subprocess and exposes a simple completion API."""

    def __init__(
        self,
        model_path: str,
        server_binary: str = "/home/ai/notebook/llama.cpp/build/bin/llama-server",
        host: str = "127.0.0.1",
```

```

port: int = 8080,
n_gpu_layers: int = 0,
threads: int = 4,
ctx_size: int = 4096,
n_parallel: int = 1,
):
    self.model_path = model_path
    self.server_binary = server_binary
    self.host = host
    self.port = port
    self.n_gpu_layers = n_gpu_layers
    self.threads = threads
    self.ctx_size = ctx_size
    self.n_parallel = n_parallel
    self.base_url = f"http://{host}:{port}"
    self._process: subprocess.Popen | None = None
    self._log_thread: threading.Thread | None = None

```

Parameter	Default	Description
model_path	(required)	Path to the .gguf model file.
server_binary	/home/ai/...	Path to the compiled llama-server binary.
host	127.0.0.1	Interface to bind; use 0.0.0.0 to allow remote access.
port	8080	TCP port the server will listen on.
n_gpu_layers	0	Number of transformer layers to offload to GPU. Set 99 to use all layers.
threads	4	Number of CPU threads for generation.
ctx_size	4096	Context window size in tokens.
n_parallel	1	Number of simultaneous inference slots.

9.2 Lifecycle Methods

Three methods control the server process. `start()` and `stop()` can be called manually, or the class can be used as a context manager (recommended).

```

def start(self, timeout: int = 120):
    """Start llama-server and wait until it is ready."""
    if not os.path.exists(self.server_binary):
        raise FileNotFoundError(f"llama-server binary not found: {self.server_binary}")
    if not os.path.exists(self.model_path):
        raise FileNotFoundError(f"Model file not found: {self.model_path}")

    cmd = [
        self.server_binary,
        "-m", self.model_path,
        "--host", self.host,

```

```

        "--port", str(self.port),
        "-t", str(self.threads),
        "-c", str(self.ctx_size),
        "--parallel", str(self.n_parallel),
        "--log-disable",          # keep our stdout clean
    ]
    if self.n_gpu_layers > 0:
        cmd += ["-ngl", str(self.n_gpu_layers)]

    print(f"Starting llama-server on {self.base_url} ...")
    self._process = subprocess.Popen(
        cmd,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        text=True,
    )

    # Drain server output on a daemon thread so the pipe never blocks
    self._log_thread = threading.Thread(target=self._drain_logs, daemon=True)
    self._log_thread.start()

    self._wait_until_ready(timeout)
    print("llama-server is ready.\n")

    def stop(self):
        """Gracefully stop the server."""
        if self._process and self._process.poll() is None:
            print("\nStopping llama-server ...")
            self._process.send_signal(signal.SIGTERM)
            try:
                self._process.wait(timeout=10)
            except subprocess.TimeoutExpired:
                self._process.kill()
            print("llama-server stopped.")

    def __enter__(self):
        self.start(); return self
    def __exit__(self, *_):
        self.stop()

```

9.3 Inference — completion()

Sends a single prompt to the /completion endpoint and returns the generated text as a plain string.

```

def completion(
    self,
    prompt: str,
    max_tokens: int = 256,
    temperature: float = 0.7,
    top_p: float = 0.9,
    stop: list[str] | None = None,
) -> str:
    """Send a /completion request and return the generated text."""
    payload = {

```

```

        "prompt":      prompt,
        "n_predict":   max_tokens,
        "temperature": temperature,
        "top_p":       top_p,
        "stop":        stop or ["<|user|>", "<|system|>"],
        "stream":      False,
    }
    resp = requests.post(
        f"{self.base_url}/completion",
        json=payload,
        timeout=300,
    )
    resp.raise_for_status()
    return resp.json()["content"].strip()

```

9.4 Internal Helpers

Two private methods keep the server process healthy during startup and runtime.

```

def _drain_logs(self):
    """Read and discard server log output so the pipe never fills."""
    for _ in self._process.stdout:
        pass

def _wait_until_ready(self, timeout: int):
    deadline = time.time() + timeout
    while time.time() < deadline:
        if self._process.poll() is not None:
            raise RuntimeError("llama-server exited unexpectedly during
startup.")
        try:
            r = requests.get(f"{self.base_url}/health", timeout=2)
            if r.status_code == 200:
                return
        except requests.ConnectionError:
            pass
        time.sleep(1)
    self._process.kill()
    raise TimeoutError(f"llama-server did not become ready within {timeout}s.")

```

 **Note:** `_drain_logs()` runs on a daemon thread, which means it is automatically killed when the main process exits — no cleanup needed.

9.5 Usage Example

Use the class as a context manager so the server is guaranteed to stop even if your script raises an exception.

```
from eval_socraticlm import LlamaServer

with LlamaServer(
    model_path="/home/ai/notebook/Qwen3.5-35B-A3B-MXFP4_MOE_BF16.gguf",
    server_binary="/home/ai/notebook/llama.cpp/build/bin/llama-server",
    host="127.0.0.1",
    port=8080,
    n_gpu_layers=99,      # offload all layers to GPU
    ctx_size=8192,       # 8 k context window
) as server:
    # Server starts automatically; stops when the with-block exits
    reply = server.completion(
        "What is Socratic questioning?",
        max_tokens=200,
        temperature=0.7,
    )
    print(reply)
```

 **Tip:** The LlamaServer class pings /health every second until the server is ready, so your script will never run inference before the model is fully loaded.

Generated from eval_socraticlm.py • AI Lab • 2025